Week 8 Lab Handout

API Document — Publishing Real Docs and a Blind Test

Time: 110 minutes

Bring: your laptop, your working API and `openapi.yaml` from Weeks 4–7, and a teammate ready to swap README files with a team outside your ring

---

Before you start: docs are a product, not an afterthought

> Every check you've run so far — your own tests in Week 7, your partner's tests against your contract — assumed the person running them already knew how your API worked. Today that assumption goes away.

The real test of documentation isn't whether it's accurate. It's whether someone who has never spoken to you, and never will, can read it once and get a working request out the other side. If the honest answer is “no, they'd have to message me,” the docs aren't done — no matter how much you wrote.

That's why today ends with a blind test, not a checklist. A README you're confident in and a README that actually works for a stranger are two different claims, and only one of them gets checked today.

Part A — Generate Your Docs Site (25 min)

1. Take your `openapi.yaml` from Week 4 (updated with any changes since) and run it through **Swagger UI** or **Redoc** to render it as a browsable docs page.
2. Confirm every endpoint in your contract actually appears on the rendered page — if one is missing, your YAML has a structural error worth fixing now, not during the blind test.
3. Either host the rendered output (GitHub Pages or similar) or commit the rendered HTML/build output directly to your repo — either is fine, as long as a teammate can open it without running anything themselves.
4. Click through your own rendered docs as if you were a stranger. Does every endpoint show a request shape and a response shape, or just one of the two?

Part B — Write Your README Quickstart (35 min)

Your rendered docs page from Part A shows the *shape* of your API. Your `README.md` has to show someone how to actually *use* it, in order, from zero.

What your README must include

- **Overview** — one paragraph: what this API is, and who it's for
- **Auth instructions** — exactly how to authenticate, even if it's a placeholder for now (real auth arrives in Week 9)
- **One working example per major endpoint** — a real request AND the real response it returns, not just a description
- **Error format reference** — what a failed response actually looks like, so a caller can tell success from failure without guessing

- **How to run the API locally** — the exact commands, assuming a completely clean clone (this is `run.sh` again, but for a reader instead of the autograder)

Bad vs. good — this is what gets checked in Part D

Bad (leaves you guessing)	Good (copy, paste, it works)
"Returns a task by ID."	GET /tasks/42 → 200 { "id": 42, "title": "Write README", "dueDate": "2026-10-01" }
"Send the correct fields to create a task."	POST /tasks with body { "title": string, "dueDate": date } → 201 with the created object
(no error example given)	GET /tasks/9999 → 404 { "message": "Task not found" }

Both columns describe the same real endpoints. Only one of them lets a stranger act without messaging you first — that's the entire bar today.

Write one real quickstart command

Pick your single most important endpoint and write one command a stranger could literally copy and paste, right now, and get a real response:

```
curl -H "Authorization: Bearer <token>" https://your-api-url/tasks
```

Run it yourself, from a clean terminal, exactly as written in your README. If it doesn't work as written, a stranger's copy-paste won't either — fix it now.

Part C — The Blind Swap (15 min)

1. Find a team that is **not** your ring partner and has **never worked with your API before** — the whole point is zero shared context.
2. Send them your repo's `README.md` (and a way to reach your running API — a shared URL, or instructions to run it locally) and nothing else. No verbal walkthrough, no "let me just explain one thing first."
3. Receive the same from them: their `README.md`, no explanation.

If you can't find a team outside your ring during lab (scheduling gap), pair with whichever team is available and flag the substitution in your Part E report — the exercise still works as long as the reader genuinely has no prior context on that API.

Part D — Run the Blind Test (20 min)

Using **only** the README you just received, attempt to make **one successful call** against that team's live API. Rules:

- No messaging them, no asking questions, no peeking at their code.
- If you get stuck, note exactly where and why — that's the most useful data point you'll produce all week.
- Keep going until you either succeed or genuinely cannot proceed with what the README gave you.

While you wait for the team testing *your* README, watch what they attempt (without helping) — seeing exactly which line of your own README they misread or skipped is worth more than any self-review would

have told you.

Part E — Finalize and Report (15 min)

Create `BLIND_TEST_REPORT.md` at your repo root, documenting:

1. **What happened when your README was tested** — did the other team succeed? How long did it take? Where (if anywhere) did they get stuck?
2. **What you fixed** in your docs as a result, or what you're flagging as a gap you didn't have time to close today.
3. One honest paragraph: if the other team never got a working response, say so plainly — "it didn't work" is a real and useful result, not a failure to report.

Commit and push everything:

```
git add . README.md BLIND_TEST_REPORT.md
git commit -m "Week 8: published docs, README quickstart, blind test"
git push
```

This week's deliverable checklist

- Docs site generated from `openapi.yaml` (Swagger UI or Redoc), rendering every endpoint
- `README.md` with overview, auth placeholder, one example per major endpoint, error reference, and local run instructions
- README swapped with a team outside your ring, with no verbal explanation given
- Blind test actually run against a real, live API — not a thought experiment
- `BLIND_TEST_REPORT.md` written honestly, including "it didn't work" if that's what happened
- All files committed and pushed

Rubric (20 pts)

Criterion	Points
Docs site renders correctly and completely	6
README quickstart works when followed literally, by someone else	8
Blind test performed and reported honestly	6

Common problems and quick fixes

Problem	Fix
Our docs site is missing an endpoint	Check your openapi.yaml for a syntax error near that path — a missing colon or bad indent silently drops the whole block
The other team got stuck immediately, on step one	That's real signal, not bad luck — re-read your own "how to run locally" section as if you'd never seen your code before

Problem	Fix
We ran out of time before the blind test	Do Part D asynchronously right after lab and update BLIND_TEST_REPORT.md before the deliverable deadline — don't skip the swap just because the clock ran out
Our quickstart command works for us but not for them	You likely have local state (a seeded task, a cached token) they don't — test your own quickstart from a truly clean environment
We're not sure if a failure is “their fault” or “our docs' fault”	If a reasonable person could follow your README exactly as written and still get stuck at that point, it's a docs gap — fix it, don't argue the point

Next week: your docs are public and correct — now anyone who can read them can also try to abuse them. Security begins.